

HPL-AI to cuTile Porting Report

From the C Reference Path to Python and Quotient-Based GPU Kernels

cuTile analysis workspace

June 2026

Abstract

This report documents the local HPL-AI extraction in `hpl_ai/cutile_hpl_ai.py` as a two-step port. The first step is an executable Python port of the compact C reference path: matrix and right-hand-side generation, precision conversion, no-pivot LU factorization, triangular solve, GMRES iterative refinement, and the scaled residual check. The second step is a narrower cuTile port that maps the dense, regular, low-level arithmetic kernels to GPU tile notation while leaving generation, refinement, and validation on the host. The intent is not to claim that every part of the benchmark has been made faster on the GPU. The intent is to make the numerical decisions explicit, preserve the benchmark semantics that matter for residual quality, and create compact kernels that are good targets for NixNan floating-point diagnostics.

Contents

1	Purpose and Scope	3
2	Reading <code>hpl_ai/cutile_hpl_ai.py</code> as an HPL-AI Port	3
3	First Port: C Reference Algorithm to Python	5
3.1	Matrix and RHS Generation	5
3.2	No-Pivot LU	6
3.3	Triangular Solve	6
3.4	GMRES and Residual	6
4	Second Port: Python Extraction to cuTile	7
4.1	Conversion Kernels	7
4.2	LU Kernels	7
4.3	Triangular-Solve Kernels	7
4.4	Launch Structure	8
5	Quotient-Based Iteration Mapping	8
6	Host/GPU Boundary and Rationale	9
7	Precision Choices as an Engineering Contract	10
7.1	Input Precision	10
7.2	Factor and Solve Precision	10
7.3	Refinement and Residual Precision	11
8	Precision Policy as Condition Number Increases	11

9 Verification and Diagnostic Use	12
10 Limitations and Next Iterations	12
11 Conclusion	13

1 Purpose and Scope

The local analysis directory already contains `cuTileGenDoc.md`, which is a thorough, function-by-function note on the extraction. This document is a more reader-oriented report intended to become a 12–14 page PDF. It starts from `hpl_ai/cutile_hpl_ai.py`, treats the file as the central artifact, and records how the program moves from an upstream C benchmark path to Python and then to cuTile notation.

The upstream source identified by the existing notes is the Innovative Computing Laboratory HPL-AI reference implementation [2]. HPL-AI itself sits in the same benchmark family as HPL, the High Performance Linpack benchmark [6], but uses mixed-precision linear algebra and iterative refinement to expose modern accelerator arithmetic [5]. The local file `hpl_ai/cutile_hpl_ai.py` is therefore best read as a port of a specific benchmark path rather than as a general linear-solver library.

Figure 1 summarizes the end-to-end story. The important division is that the Python port keeps the full algorithmic path executable, while the cuTile port selects the GPU-sized arithmetic pieces that are compact enough to inspect and diagnostic enough for NixNan.

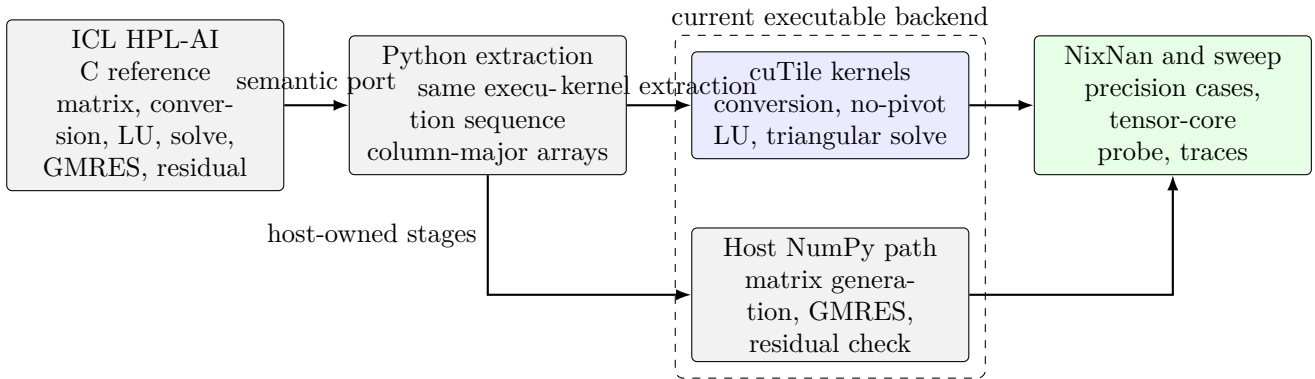


Figure 1: End-to-end flow from the upstream C reference path to the local Python extraction, cuTile kernels, and diagnostic sweep artifacts.

Two qualifications are useful at the start. First, the local port intentionally keeps the HPL-style column-major storage model. This avoids hidden transpositions when comparing the Python code, the cuTile kernels, and the C reference. Second, the GPU mapping is selective. It is based on regularity, data-parallel density, diagnostic value, and implementation risk. It is not a blanket assertion that every operation in the full benchmark is better on a GPU.

2 Reading `hpl_ai/cutile_hpl_ai.py` as an HPL-AI Port

The first evidence is explicit in the module docstring and the local license file. `hpl_ai/cutile_hpl_ai.py` states that it is derived from the ICL HPL-AI reference implementation and lists the extracted path: generate the matrix and RHS, convert inputs, factor with no-pivot LU, solve the triangular factors, convert the solution for refinement, optionally run GMRES, and compute the scaled residual. The local `hpl_ai/HPL_AI_LICENSE` keeps the upstream license note.

The second evidence is structural. The Python functions are not independent inventions with similar names. They mirror the reference benchmark’s dataflow: `hpl_ai_matrix` and `make_rhs` cover the generator path, `lu_nopiv_cpu` covers no-pivot LU, `lu_solve_cpu` covers the forward and backward triangular solve, `gmres_refine` covers left-preconditioned refinement, and `scaled_residual` covers the final HPL-style acceptance metric.

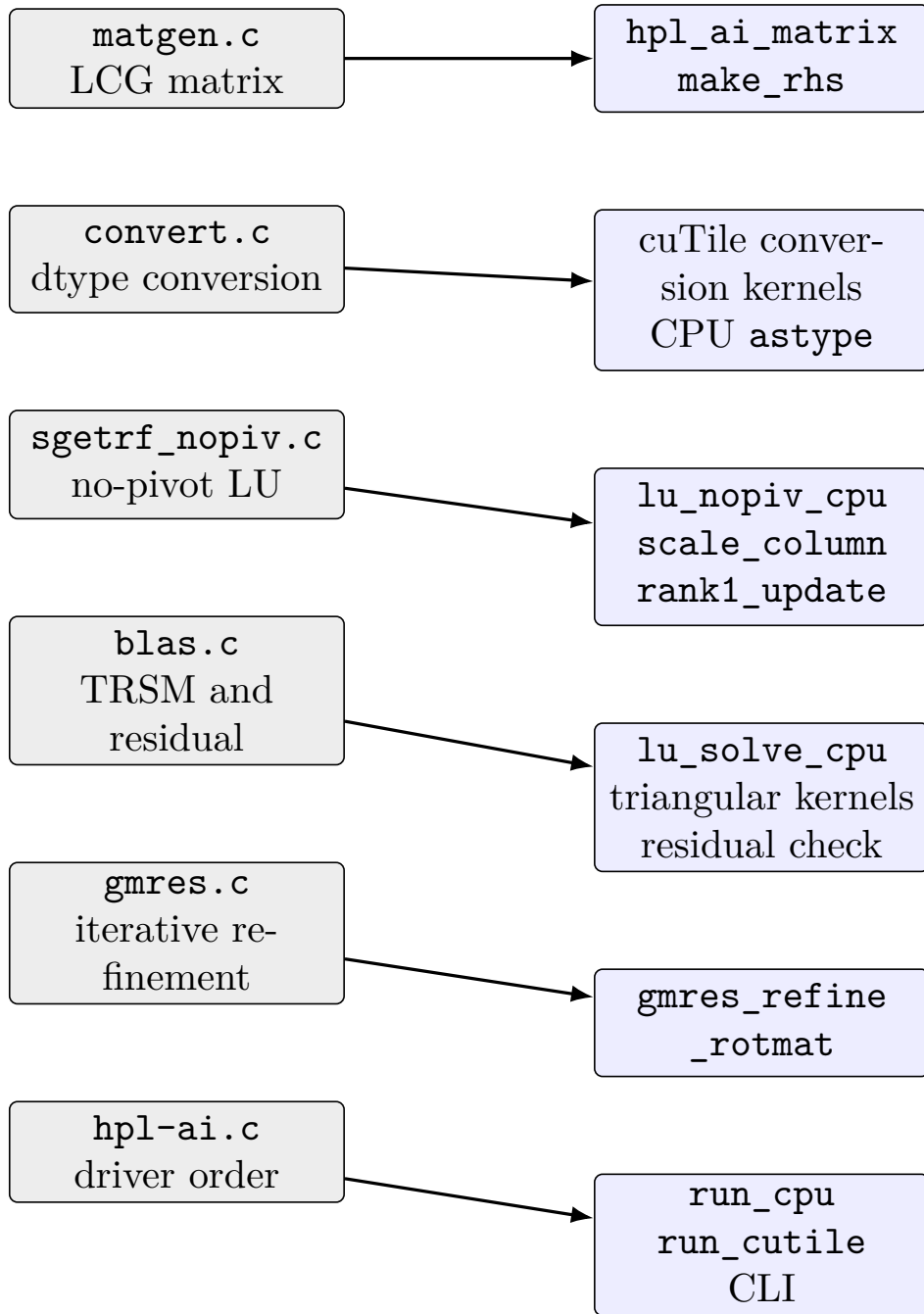


Figure 2: Source-to-port correspondence used in the local notes. The port changes implementation form, but preserves the benchmark sequence.

The C reference is compact, but it still carries several concerns at once: storage layout, generator determinism, precision staging, factorization, solve, refinement, and residual validation. The Python extraction deliberately separates these concerns into small functions and dataclasses. This is the first major design decision in the port. Instead of starting with the GPU, the code first creates a host executable model whose behavior can be compared against the cuTile backend.

The `PrecisionConfig` dataclass is central to that separation. The reference benchmark has a mixed-precision intent, but the local analysis needs to sweep multiple precision choices. The port therefore records five roles: input precision, factorization precision, solve precision, refinement precision, and residual precision. This keeps the benchmark default visible while allowing the sweep to stress the low-precision stages.

The `MatrixConfig` dataclass is the analogous separation for matrix generation. The upstream-like matrix kind is preserved as `hpl-ai`. A second kind, `conditioned`, is added for controlled condition-number experiments. This addition is explicitly not an upstream feature. It is an analysis feature that lets the precision policy be studied under increasing numerical stress.

3 First Port: C Reference Algorithm to Python

The first port converts the reference algorithm into readable Python while preserving the algorithmic order and the memory indexing convention. It uses NumPy arrays for the CPU oracle and records the exact flat column-major address calculation:

$$\text{index}(i, j, n) = i + jn.$$

This is the Python form of the reference macro pattern that stores $A(i, j)$ at `row + col * lda`. Keeping that layout avoids a subtle class of bugs. If the CPU path used row-major arrays and the cuTile path used column-major buffers, comparing them would require additional layout translations, and those translations could hide porting errors.

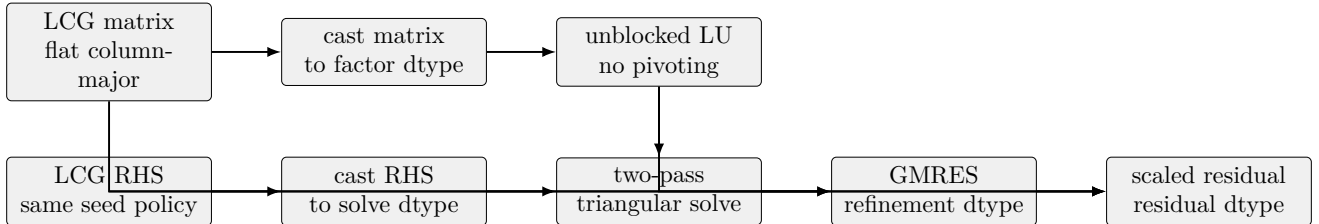


Figure 3: Python extraction data path. The host reference owns the full numerical sequence and is used as the executable oracle.

3.1 Matrix and RHS Generation

The local generator is intentionally deterministic. The function `_lcg_values` uses the same essential shape as the C generator: a 64-bit linear congruential generator, uint64 wraparound, and a scale factor that produces values near the interval $[-0.5, 0.5)$. The generated matrix is then interpreted as an $n \times n$ column-major matrix. The diagonal entries are replaced by the sum of the off-diagonal absolute values in the same row. The resulting matrix is row diagonally dominant, which is one of the reasons a no-pivot LU path can be used as a benchmark path rather than as a general purpose solver.

The right-hand side uses the same generator helper with a related seed. The port keeps generation on the host because generation is not the diagnostic target of this analysis. It also keeps the generator easy to compare with the C reference. Moving the generator to a GPU kernel would add random-state

and layout surface area without improving the core question: whether the extracted mixed-precision arithmetic kernels can be expressed compactly in cuTile and observed by NixNan.

3.2 No-Pivot LU

The upstream C reference uses no-pivot LU in a blocked or recursive style. The Python extraction uses the mathematically equivalent unblocked outer-product form:

$$\begin{aligned} A_{k+1:n,k} &\leftarrow A_{k+1:n,k} / A_{k,k}, \\ A_{k+1:n,k+1:n} &\leftarrow A_{k+1:n,k+1:n} - A_{k+1:n,k} A_{k,k+1:n}. \end{aligned}$$

This is a deliberate porting choice. The blocked schedule is better aligned with high-performance GEMM use, but the unblocked form exposes the two operations that need to become cuTile kernels: scale a pivot column and apply a rank-1 update to the trailing matrix.

That choice trades some performance potential for clarity. In this local analysis, clarity is valuable because the goal is to create a small GPU surface that can be instrumented and reasoned about. The numerical operation is still no-pivot LU. The scheduling strategy changed.

3.3 Triangular Solve

After factorization, the Python oracle performs a unit-lower forward solve and a non-unit-upper backward solve. These loops are written in a way that mirrors the future GPU mapping. Each pivot step updates a contiguous logical region of the RHS vector. This makes it straightforward to compare the CPU version with the cuTile kernels `forward_subtract_kernel`, `scale_rhs_kernel`, and `backward_subtract_kernel`.

The triangular solve remains lower intensity than the rank-1 trailing update, but it is still mapped to cuTile because it is part of the extracted mixed-precision solve path. If it remained only on the host, the cuTile backend would no longer represent the complete low-precision factor-and-solve phase. Keeping it on the GPU also avoids an unnecessary transfer between LU factorization and the initial solution.

3.4 GMRES and Residual

The GMRES routine follows the standard generalized minimal residual structure [4]: build Krylov vectors, orthogonalize, apply Givens rotations, solve the small projected problem, and update the solution. In the local code it is left-preconditioned by the LU factors. The residual check uses the HPL-style scaled infinity-norm expression:

$$\frac{\|Ax - b\|_\infty}{\epsilon(\|A\|_\infty\|x\|_\infty + \|b\|_\infty)n}.$$

The threshold is the local constant `RESIDUAL_THRESHOLD = 16.0`.

Both GMRES and the residual check remain on the host in the current port. This is not because they are unimportant. They are important to benchmark validity. They stay on the host because they are not the smallest useful cuTile surface. GMRES contains reductions, vector orthogonalization, small dense solves, and control decisions. A high-quality GPU GMRES implementation would require a separate design. The current report therefore treats host GMRES as the validation and refinement layer around a GPU low-precision solve.

4 Second Port: Python Extraction to cuTile

The second port extracts the dense arithmetic operations from the Python model and maps them to cuTile kernels. The resulting backend is `run_cutile`. It still generates matrix data on the host, but allocates CuPy buffers, launches cuTile kernels for the factor-and-solve path, copies the initial solution back, and optionally runs host GMRES.

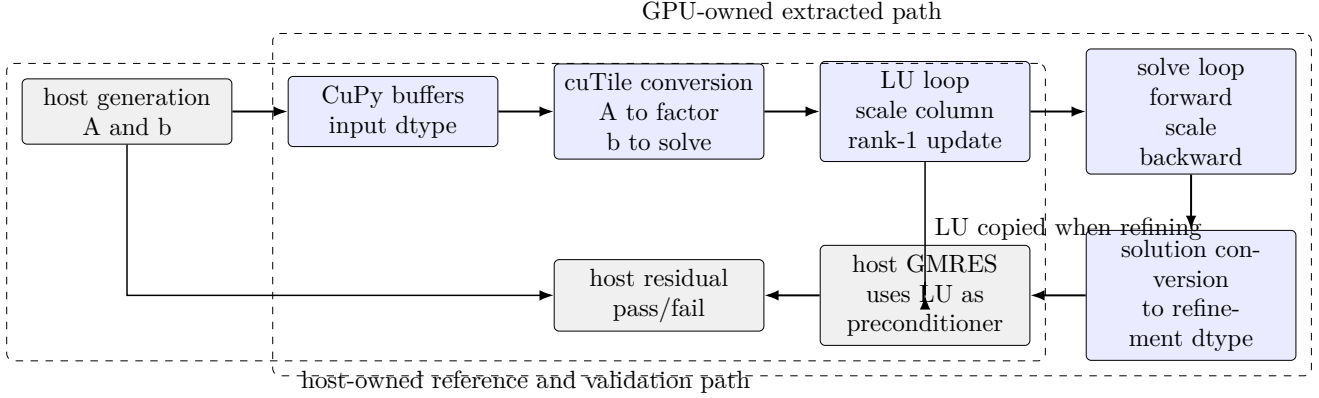


Figure 4: cuTile backend execution. The GPU owns conversion, no-pivot LU, and triangular solve; the host owns generation, refinement, and final validation.

4.1 Conversion Kernels

The conversion kernels are intentionally simple. Each kernel loads a tile of width $W=128$, casts it to the requested target dtype, masks inactive lanes beyond the logical total, and stores the result. There are three concrete kernels for `float16`, `float32`, and `float64`. The helper `_conversion_kernel` chooses among them based on the precision role currently being exercised.

Mapping conversion to cuTile is useful for two reasons. First, conversion is part of the mixed-precision benchmark sequence. Second, it makes the precision boundary explicit in the GPU trace. A run with `factor_precision = float16` and one with `factor_precision = float32` do not merely change a host dtype; they launch different conversion paths into GPU storage.

4.2 LU Kernels

The LU factorization becomes two kernels per pivot step. The first, `scale_column_kernel`, divides each entry below the pivot by the pivot. The second, `rank1_update_kernel`, updates the trailing matrix. The rank-1 update is the largest regular arithmetic region in the unblocked LU algorithm. It is therefore the clearest candidate for GPU mapping.

This mapping also makes an important performance distinction. The port does not currently use tensor-core GEMM for the LU update. The rank-1 update is scalar tile code. The separate tensor-core probe in the sweep exists to force WMMA SASS for NixNan verification, not to claim that the LU update itself is tensor-core optimized. That distinction matters for interpreting traces.

4.3 Triangular-Solve Kernels

The triangular solve becomes a sequence of vector update kernels. Forward substitution updates rows below the pivot. Backward substitution first scales the pivot entry and then updates rows above the pivot. These operations are less parallel than a full trailing matrix update because each pivot step

depends on the previous pivot result. Still, within each step the row updates are independent, and the solve belongs to the same low-precision phase as the factorization.

4.4 Launch Structure

The cuTile driver uses `_grid_for(total)` to compute a one-dimensional tile grid. A logical element id is computed from the block id and lane id:

$$\text{flat} = \text{bid}_0 W + \text{lane}.$$

Kernels use masks so extra lanes in the last tile do not write outside the logical region. This makes each kernel shape independent from exact matrix sizes. The same tile width can cover an entire matrix buffer, a trailing submatrix, or a vector segment.

5 Quotient-Based Iteration Mapping

The phrase quotient-based implementation is most concrete in the `rank1_update_kernel`. A two-dimensional trailing matrix is represented as one flat iteration space. Each active lane computes a flat id, then uses quotient and remainder operations to recover the logical row and column:

$$\begin{aligned} q &= \left\lfloor \frac{\text{flat}}{\text{trailing}} \right\rfloor, \\ r &= \text{flat} \bmod \text{trailing}, \\ \text{row} &= k + 1 + r, \\ \text{col} &= k + 1 + q. \end{aligned}$$

The final matrix address is still column-major:

$$\text{index} = \text{row} + \text{col} n.$$

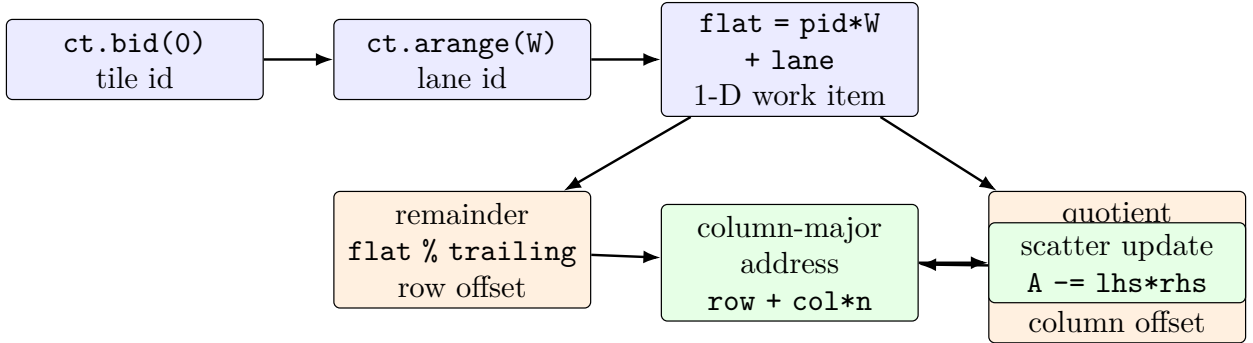


Figure 5: Quotient and remainder decomposition used to map a flat tile-lane iteration space back to trailing-matrix coordinates.

This quotient-based form has several advantages for the current port. First, it gives the kernel one simple grid dimension. The driver does not need to create a two-dimensional launch for a submatrix whose size shrinks every pivot. Second, it makes the logical operation easy to audit. Every active lane performs the same algebraic update:

$$A_{\text{row},\text{col}} \leftarrow A_{\text{row},\text{col}} - A_{\text{row},k} A_{k,\text{col}}.$$

Third, it naturally preserves the column-major memory layout. The quotient identifies the column offset, the remainder identifies the row offset, and the final address is the same expression used by the CPU oracle.

The tradeoff is that the mapping is not a tuned GEMM schedule. It is a clear rank-1 update schedule. A high-performance implementation would likely block the update, improve memory reuse, and use matrix-multiply primitives where appropriate. The current quotient-based implementation is therefore best understood as a faithful, inspectable mapping of the unblocked algorithm rather than as a final performance implementation.

6 Host/GPU Boundary and Rationale

The boundary between host and GPU is the main engineering choice in the port. Figure 6 and Table 1 summarize the decision. The GPU receives the regular factor-and-solve arithmetic. The host retains generation, conditioning experiments, GMRES refinement, and residual checking.

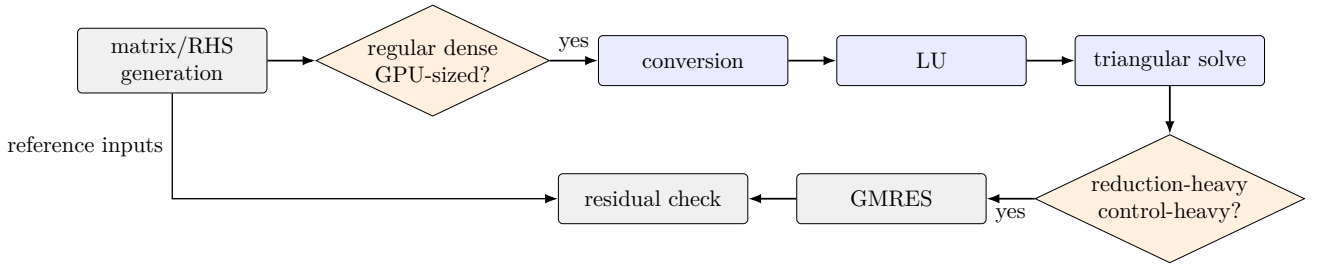


Figure 6: Boundary decision used by the current port. The decision is based on regularity, diagnostic value, and implementation risk.

Stage	Location	Rationale
LCG matrix and RHS generation	Host	Determinism and reference comparability matter more than GPU throughput. Moving it would add state-management surface with little diagnostic value.
Conditioned matrix construction	Host	QR factorization and eigenvalue-spectrum construction are analysis scaffolding, not upstream benchmark work.
Precision conversion	GPU	Conversion is part of the mixed-precision path and creates visible GPU dtype boundaries.
No-pivot LU factorization	GPU	The scale and rank-1 update steps are regular dense arithmetic and are the best compact cuTile workload seed.
Triangular solve	GPU	It completes the low-precision factor-and-solve phase and avoids transferring intermediate LU/RHS state back to the host.
GMRES refinement	Host	Orthogonalization, reductions, small solves, and convergence checks make this a larger GPU project. Host execution preserves benchmark semantics while keeping the cuTile surface compact.
Scaled residual	Host	Validation needs high precision, stable norms, and simple comparability with the CPU oracle.

Table 1: Host/GPU placement decisions in the current extraction.

The boundary was not chosen only by asking which parts might run faster on a GPU. Speedup potential matters, but it is only one criterion. The chosen GPU parts are also the parts that can be

expressed without a large runtime protocol: each lane owns one matrix or vector element update, the active set is controlled by a mask, and the kernels expose the floating-point operations that NixNan is meant to observe.

The host parts are retained because their role is different. Matrix generation defines reproducible input. The conditioned matrix path defines a numerical stress axis. GMRES protects the mixed-precision solve by applying a higher-precision correction. The residual check is the benchmark acceptance test. These stages can be GPU accelerated in a larger project, but mapping them first would make the port harder to validate.

7 Precision Choices as an Engineering Contract

The default precision configuration in `hpl_ai/cutile_hpl_ai.py` is:

```
input = float64,          factor = float32,          solve = float32,
refinement = float64,     residual = float64.
```

This is HPL-AI-like: use lower precision for the expensive factor-and-solve phase, then use higher precision for iterative refinement and validation. The choice reflects the standard mixed-precision idea that a low-precision solve can be a preconditioner for a high-precision correction, provided the problem and the low-precision factorization are stable enough [1].

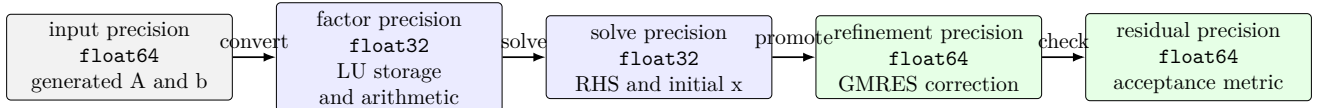


Figure 7: Precision pipeline used by the default configuration. The low-precision phase is bounded by high-precision refinement and residual validation.

7.1 Input Precision

The default input precision is `float64`. This gives the generated system a high-precision reference representation before any deliberate low-precision conversion. It also makes residual checks meaningful: the final validation can compare against the same high-precision input used by the CPU oracle.

When input precision is reduced, the experiment changes. The solver is no longer only testing low-precision factorization and solve; it is also testing whether the problem itself has been quantized. That is useful for stress studies, but it is a different question from the HPL-AI-like default.

7.2 Factor and Solve Precision

The factorization and triangular solve default to `float32`. This is the core mixed-precision compromise. It is low enough to expose accelerator-style arithmetic and higher throughput than double precision on many GPUs, while remaining much less fragile than `float16` for no-pivot LU.

The code allows `float16` factor and solve precision because the sweep is designed to expose exceptional values and precision stress. However, `float16` is not treated as the safe default for the LU path. No-pivot LU can amplify rounding and pivot-growth effects. With a difficult matrix, `float16` may overflow, underflow, or produce a singular effective factorization. Those failures are not surprising; they are useful diagnostic cases.

7.3 Refinement and Residual Precision

GMRES refinement and residual checking default to `float64`. This is the guardrail around the low-precision solve. The initial low-precision solution is not accepted by construction. It must be corrected and then pass the scaled residual threshold. Keeping this work in double precision reduces the chance that the validation logic hides the same rounding effects that the experiment is trying to study.

The residual precision is especially important. If the residual were computed in the same low precision as the factorization, a poor solve could appear less poor because the residual computation itself lost information. The local default therefore separates solve speed from validation quality.

8 Precision Policy as Condition Number Increases

Condition number changes the precision story because forward error is amplified by the sensitivity of the linear system. A rough rule is that rounding effects scale with a factor like $\kappa(A)u$, where u is the unit roundoff of the relevant arithmetic [1]. This rule is not a complete stability guarantee for no-pivot LU, but it explains why the same precision configuration can work on a benign matrix and fail on a more ill-conditioned one.

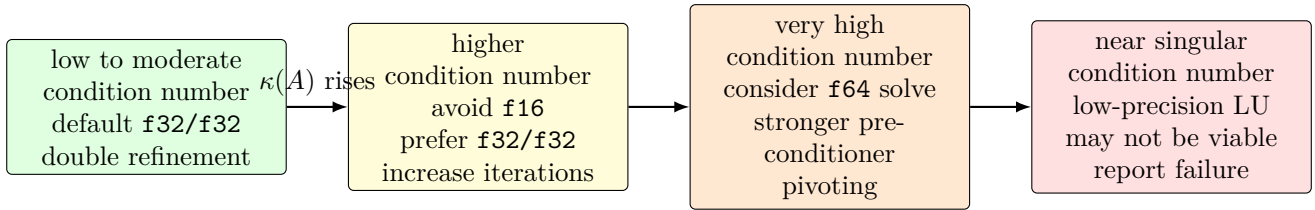


Figure 8: Qualitative precision ladder as condition number rises. The exact cutoff is problem-dependent; the direction of the policy is the important point.

Table 2 gives a practical policy for the local experiments. The bands are intentionally qualitative. They should be read as engineering guidance for sweeps, not as mathematical guarantees.

Condition regime	Factor/solve choice	Refinement/residual	Reasoning
Benign or moderate	<code>float32/float32</code> ; <code>float16</code> may be used as a stress case	<code>float64/float64</code>	The low-precision solve is likely to be a usable preconditioner, and double refinement can correct remaining error.
Rising condition number	Prefer <code>float32/float32</code> ; avoid <code>float16</code> as a production setting	Keep <code>float64/float64</code> ; allow more GMRES iterations	Sensitivity starts to magnify low-precision factorization error. Refinement needs a stronger initial solve.
High condition number	Consider <code>float64</code> solve precision or <code>float64</code> factorization for comparison	Keep <code>float64</code> ; inspect convergence and residual history	The low-precision LU factor may no longer be a reliable preconditioner. Higher precision becomes diagnostic, not just conservative.
Near singular or unstable no-pivot case	Low-precision no-pivot LU should be expected to fail	Double residual should report failure clearly	The current algorithm lacks pivoting. A robust solver would need pivoting, equilibration, or a different preconditioner.

Table 2: Suggested precision policy for condition-number sweeps in this workspace.

The most important adjustment is to promote the factor-and-solve phase before weakening valida-

tion. If the condition number goes up, the residual precision should not be reduced to make the result pass. Instead, the low-precision phase should become more accurate, or the algorithm should admit that the chosen no-pivot factorization is not adequate for that matrix.

This is also why the `conditioned` matrix family is useful. The upstream HPL-AI matrix generator does not expose a condition-number parameter. The local conditioned family creates a controlled axis for experiments: same backend, same precision roles, but increasing numerical sensitivity. That makes it possible to distinguish implementation bugs from expected precision breakdown.

9 Verification and Diagnostic Use

The local workflow has three layers of verification. First, the CPU path can run without CUDA and validates the Python extraction. Second, the cuTile backend can be compared against the CPU path with `-compare-cpu`. Third, the NixNan sweep can collect floating-point diagnostic traces around the GPU work.

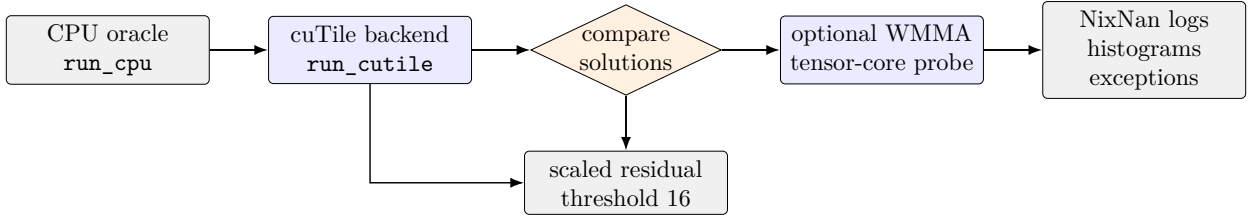


Figure 9: Verification and diagnostic layers used by the local sweep scripts.

The tensor-core probe deserves explicit separation. It is a small FP16-by-FP16 to FP32 WMMA kernel, implemented through CuPy raw CUDA code and using the CUDA WMMA interface [3]. It exists so that the NixNan run can confirm HMMA instruction instrumentation. It is not the implementation of the cuTile LU update. This separation is healthy: the trace can include tensor-core evidence without overstating what the solver kernels themselves do.

The sweep scripts record stdout, NixNan logs, run environment, and a summary table. They also expose the precision tuple:

(input, factor, solve, refinement, residual).

That tuple is the practical interface between the numerical policy described above and the diagnostic traces collected below it.

10 Limitations and Next Iterations

The current port is intentionally conservative. It provides a readable and diagnostic extraction, not a final high-performance benchmark replacement. The main limitations are as follows.

- The LU implementation is unblocked. This makes the cuTile kernels small and inspectable, but it leaves performance on the table compared with blocked GEMM-based schedules.
- The factorization is no-pivot. This matches the extracted benchmark path, but it is not a robust general solver for arbitrary matrices.
- GMRES remains on the host. A GPU GMRES port would need careful design for reductions, orthogonalization, and small projected solves.

- The conditioned matrix family is an analysis extension. It should not be confused with the upstream generator.
- The tensor-core probe is separate from the solver. It verifies instrumentation coverage for HMMA instructions, not tensor-core acceleration of the LU path.

The next useful LaTeX iteration would add line-numbered code excerpts from `hpl_ai/cutile_hpl_ai.py`, a short derivation of the scaled residual, and a table of actual sweep cases from `nixnan_sweep/data/summary.tsv`. Another useful iteration would compare the unblocked quotient-based update with a hypothetical blocked update, making clear what would need to change before pursuing performance rather than diagnostic clarity.

11 Conclusion

`hpl_ai/cutile_hpl_ai.py` is a port of the compact HPL-AI reference path into a Python executable model and then into a selective cuTile backend. The first port preserves benchmark semantics: deterministic generation, column-major layout, mixed-precision no-pivot LU, triangular solve, GMRES refinement, and scaled residual validation. The second port maps the dense low-level arithmetic into tile kernels: conversion, column scaling, rank-1 trailing update, and triangular-solve updates.

The host/GPU split is a deliberate boundary. GPU kernels are chosen where the work is regular, dense, compact, and diagnostically valuable. Host execution is kept where reference comparability, refinement correctness, and residual validation matter more than a first-pass GPU speedup. The default precision policy follows the mixed-precision benchmark idea: low precision for the factor-and-solve phase, double precision for correction and validation. As condition number rises, the policy should strengthen the low-precision phase or report failure, not weaken the residual check.

References

- [1] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, second edition, 2002.
- [2] Innovative Computing Laboratory. HPL-AI Reference Implementation. <https://bitbucket.org/icl/hpl-ai/src/main/>. Reference C implementation used by the local porting notes. Local inspection commit: 8c7ab180b3278b2b454e6e26da1f22e60463feea.
- [3] NVIDIA Corporation. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. Background for CUDA execution and WMMA tensor-core operations.
- [4] Youcef Saad and Martin H. Schultz. GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.
- [5] TOP500 Project. HPL-AI Mixed Precision Benchmark. <https://www.top500.org/hpl-ai/>. Background on the mixed-precision benchmark objective.
- [6] University of Tennessee. HPL - A Portable Implementation of the High-Performance Linpack Benchmark for Distributed-Memory Computers. <https://www.netlib.org/benchmark/hpl/>. Background for the HPL benchmark family.